

Process Memory Explorer

Aim: To develop a menu-driven C program that explores and demonstrates the major regions of a process's memory, namely Stack, Heap, BSS, Data, Read-Only Data, and Text/Code, by displaying the segment, object name, data type, value, size, and memory address.

Problem Statement: Develop a menu-driven C application called Process Memory Explorer. The program should allow the user to select different memory regions and observe the variables or objects stored in those regions.

The program should provide the following menu:

PROCESS MEMORY EXPLORER

1. Display Stack Variables
2. Display Heap Variables
3. Display BSS Variables
4. Display Data Variables
5. Display Read-Only Data
6. Display Function Addresses
7. Display Individual Memory Bytes
8. Display Complete Memory Layout
9. Test NULL Pointer Safety
10. Exit

Algorithm

1. Start the program.
2. Declare initialized global variables for the Data segment.
3. Declare uninitialized global variables for the BSS segment.
4. Declare a string literal for the Read-Only Data section.
5. Define functions for demonstrating the Text/Code segment.
6. Inside the required functions, declare local variables to demonstrate the Stack.
7. Dynamically allocate memory using malloc() to demonstrate the Heap.
8. Display the menu to the user.
9. Read the user's choice.
10. Perform the selected operation:
 - o Display stack variables and their addresses.
 - o Display heap variables and their addresses.
 - o Display BSS variables and their addresses.
 - o Display initialized Data variables and their addresses.
 - o Display the address and contents of the string literal.
 - o Display function addresses.
 - o Display individual bytes of a selected object.
 - o Display the complete conceptual memory layout.
 - o Demonstrate safe checking of a NULL pointer.
11. Repeat the menu until the user selects Exit.
12. Release dynamically allocated memory using free().
13. Set the heap pointer to NULL.
14. Stop the program.

Output:

```
=====
PROCESS MEMORY EXPLORER
=====
```

1. Stack
2. Heap
3. BSS
4. Data
5. Read-Only Data
6. Function Addresses
7. Individual Memory Bytes
8. Complete Memory Layout
9. NULL Pointer Safety
10. Exit

Enter choice: 1

STACK

a = 10, Address = 0x7ffd1234

b = 20.50, Address = 0x7ffd1230

```
=====
PROCESS MEMORY EXPLORER
=====
```

1. Stack
2. Heap
3. BSS
4. Data
5. Read-Only Data
6. Function Addresses
7. Individual Memory Bytes
8. Complete Memory Layout
9. NULL Pointer Safety
10. Exit

Enter choice: 2

HEAP

p[0] = 50, Address = 0x55a81230

p[1] = 100, Address = 0x55a81234

```
=====
PROCESS MEMORY EXPLORER
=====
```

1. Stack
2. Heap
3. BSS
4. Data
5. Read-Only Data

6. Function Addresses
7. Individual Memory Bytes
8. Complete Memory Layout
9. NULL Pointer Safety
10. Exit

Enter choice: 3

BSS

bssInt = 0, Address = 0x55a81020

bssFloat = 0.00, Address = 0x55a81024

PROCESS MEMORY EXPLORER

1. Stack
2. Heap
3. BSS
4. Data
5. Read-Only Data
6. Function Addresses
7. Individual Memory Bytes
8. Complete Memory Layout
9. NULL Pointer Safety
10. Exit

Enter choice: 4

DATA

globalInt = 100, Address = 0x55a81010

globalFloat = 25.50, Address = 0x55a81014

PROCESS MEMORY EXPLORER

1. Stack
2. Heap
3. BSS
4. Data
5. Read-Only Data
6. Function Addresses
7. Individual Memory Bytes
8. Complete Memory Layout
9. NULL Pointer Safety
10. Exit

Enter choice: 5

READ-ONLY DATA

Message = C Programming
Address = 0x55a80010

PROCESS MEMORY EXPLORER

1. Stack
2. Heap
3. BSS
4. Data
5. Read-Only Data
6. Function Addresses
7. Individual Memory Bytes
8. Complete Memory Layout
9. NULL Pointer Safety
10. Exit

Enter choice: 6

TEXT / CODE

function1 = 0x55a7f150
function2 = 0x55a7f180

PROCESS MEMORY EXPLORER

1. Stack
2. Heap
3. BSS
4. Data
5. Read-Only Data
6. Function Addresses
7. Individual Memory Bytes
8. Complete Memory Layout
9. NULL Pointer Safety
10. Exit

Enter choice: 7

MEMORY BYTES

Byte 0 = 244, Address = 0x7ffd1200
Byte 1 = 1, Address = 0x7ffd1201
Byte 2 = 0, Address = 0x7ffd1202
Byte 3 = 0, Address = 0x7ffd1203

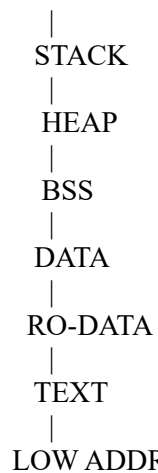
PROCESS MEMORY EXPLORER

1. Stack
2. Heap
3. BSS
4. Data
5. Read-Only Data
6. Function Addresses
7. Individual Memory Bytes
8. Complete Memory Layout
9. NULL Pointer Safety
10. Exit

Enter choice: 8

MEMORY LAYOUT

HIGH ADDRESS



LOW ADDRESS

PROCESS MEMORY EXPLORER

1. Stack
2. Heap
3. BSS
4. Data
5. Read-Only Data
6. Function Addresses
7. Individual Memory Bytes
8. Complete Memory Layout
9. NULL Pointer Safety
10. Exit

Enter choice: 9

NULL POINTER TEST

Pointer is NULL. Do not dereference it.

PROCESS MEMORY EXPLORER

1. Stack
2. Heap
3. BSS
4. Data
5. Read-Only Data
6. Function Addresses
7. Individual Memory Bytes
8. Complete Memory Layout
9. NULL Pointer Safety
10. Exit

Enter choice: 10

Program ended.